



JC4002 Knowledge Representation

Day 3: Logic Programming

Dr Ferdian Jovan

ferdian.jovan@abdn.ac.uk

Dr Aiden Durrant

aiden.durrant@abdn.ac.uk

Dr Jiangtian Nie

jiangtian.nie@abdn.ac.uk

Day 3

Logic Programming

1. Introduction
2. Paradigm
3. Options

Logic Programming

- Programming based on formal (symbolic) logic and logical inference
- Declarative programming
 - What is desired from the program
 - Statement of facts and propositions that must be satisfied
 - Specify knowledge (facts) and how the knowledge can be applied (rules)
 - Programming environment uses built-in procedures to reason about the knowledge and apply resolution and unification for proving statements or answering questions
- Very different from traditional procedural languages (based around procedures with control constructs, such as OOP)

Application of Logic Programming

- parsing
- relational database management systems
- expert systems
- natural language processing
- symbolic equation solving
- planning
- explainability
- proof systems
- programming language implementation

Day 3

Logic Programming

1. Introduction
- 2. Paradigm**
3. Options

Common Syntax for Logic Programming

- ❑ Program are sets of sentences written in logical form
- ❑ These sentences express facts and rules about a particular problem domain
- ❑ Facts and rules are written in the form of clauses
- ❑ Variables are uppercase, everything else is lowercase
- ❑ A rule (headed horn clause) is written as:

$$\begin{array}{ccc} \text{consequent} & & \text{antecedent} \\ \underbrace{H} & \text{:-} & \underbrace{T_1, T_2, \dots, T_n.} \end{array}$$

- ❑ H is the head and $T_1, T_2, \dots, T_n$ is the body
- ❑ Which reads as $T_1, T_2, \dots, T_n \rightarrow H$
- ❑ Facts are rules with no body: H .
- ❑ All clauses end with a dot (.)

Syntax Example

☐ Facts:

- man(john).
- woman(alice).
- rich(john).
- marriedto(john,alice).

☐ Predicates:

- ☐ man, woman, marriedto

☐ Atoms:

- ☐ john, alice

☐ Rules:

- ☐ loves(X , alice) :- rich(X), man(X).
- ☐ loves(X , john) :- woman(X), not (X = alice).

Interpreter

- ❑ In a compiled language the program is directly translated (e.g. C, C++, Java, Rust)
- ❑ In an interpreted language the interpreter reads and executes the program (Python, Prolog, Ruby)
- ❑ Assume the facts and rules from previous slides are loaded into a program
- ❑ We can now ask queries/goals:

Example

- ?- man(john).
true
- ?- man(alice).
false
- ?- loves(X,alice).
X = john

Example Procedural Interpretation

- ❑ All logic programs have a declarative reading (execution based on proof procedure)
- ❑ Some logic languages also have procedural interpretation for performing goal-reduction (i.e., recursive resolution, such as Prolog): to solve H we solve T_1 and solve . . . and solve T_n
- ❑ Example clause (fact): `man(john).`
 - Procedure that returns john is a man
 - Procedure to find an X that is man which will assign john to X (along with any other man in the KB)
- ❑ Example clause (rule): `loves(X , alice) :- rich(X ), man(X).`
 - Procedure to test if X loves alice by testing if X is rich and X is a man
 - Procedure to find an X that loves alice by finding an X that is rich and a man

Day 3

Logic Programming

1. Introduction
2. Paradigm
- 3. Options**

Existing Languages

☐ Prolog

- Declarative reading and procedural interpretation

☐ Datalog

- Declarative reading
- Subset of Prolog
- Useful for data integration, machine learning, etc.

☐ Mercury

- Declarative reading
- Subset of Prolog (compiled rather than interpreted)
- Software engineering principles

☐ Answer Set Programming (ASP)

- Declarative reading
- Difficult search problems
- Always terminates

☐ Constraint logic programming

- Combines Horn clauses with constraint solving techniques

☐ Concurrent logic programming

- Mixes logic programming with concurrent programming

Summary

- What is logic programming and what it is useful for
- Standard syntax
- Some examples
- Existing languages

Until the next session:

- Install SWI-Prolog to your computer.